← Back to calendar

On this page

73% progress of Week 4

Mandatory deliverable

LAB | Chaos agent: tools, lore, questionable judgment Lab

Learning Goal

You will build or evaluate one NormalObjects - Creative Complaint Handler (LangChain) path and prove the answer is grounded in inspectable evidence.

Working Context

This lab focuses on the chain between source material, tool behavior, and final answer. You keep the system small enough to trace one query, one retrieval or tool step, and one final output. The proof should show grounding, not just a successful response.

Proof of Learning

Submit one `lab_proof.md` with the code or workflow, one query, retrieved or tool evidence, final output, and one failure or limitation.

Pseudocode

Copy Explain

```
INPUT: source documents, tool result, or user query
DECIDE: retrieval, tool, agent, or guardrail path
BUILD: prepare source → run path → capture output
VERIFY: inspect evidence that supports the answer
EXPLAIN: identify one grounding risk or failure mode
```

Prompt Discipline

Use AI for review, debugging, or critique. Do not ask it to produce the final artifact or final answer.

Copy Explain

```
Role: Retrieval or agent reviewer
Task: Review my plan for grounding and verification.
Inputs: source material, query, tool path, and proof evidence.
Constraints: Do not produce the final answer. Identify weak grounding and missing traces.
Output format: three grounding risks, two trace checks, and one simplification.
Verification: Name the evidence that ties the answer to the source.
```

Build

Step-by-Step Instructions

Step 1: Setup and Project Structure

Do: Set up your environment and create the project structure.

Do:

1. Create a new file: `normalobjects_langchain.py`
2. Install required packages
3. Set up OpenAI API key

First command:

```
pip install langchain langchain-openai python-dotenv
```

Copy Explain

Expected first output: package installation completes and your script runs without import errors.

Tip: if you are not using the `os` library, this code will need to be adapted.

Tip: if you are not using `AgentExecutor`, this code will need to be adapted.

Tip: if you are not using `create_openai_tools_agent`, this code will need to be adapted.

Tip: if you are not using `ChatPromptTemplate`, this code will need to be adapted.

Tip: if you are not using `MessagesPlaceholder`, this code will need to be adapted.

Tip: if you are not using `List` and `Dict` from `typing`, this code will need to be adapted.

Important:

Tip: The correct modern imports for this lab are:

```
from langchain_openai import ChatOpenAI
from langchain.agents import create_agent
from langchain_core.tools import tool
from langchain_core.prompts import ChatPromptTemplate,
MessagesPlaceholder
```

Copy Explain

An older version of the code is also possible:

```
from langchain_openai import ChatOpenAI
from langchain_classic.agents import AgentExecutor,
create_openai_tools_agent
from langchain_core.tools import tool
from langchain_core.prompts import ChatPromptTemplate,
MessagesPlaceholder
```

Copy Explain

The `langchain_classic` is not the ideal package: we recommend do not use it since it is an in between solution.

Tip: in modern LangChain examples, many primitives come from `langchain_core` (for example tools, messages, or runnables), so adapt your imports if you copy code from notebooks.

Step 2: Create Creative Tools

Do: Define themed tools that the agent can use creatively.

Do:

1. Create tools for consulting different "sources" in the Normal Objects universe
2. Make tools flexible and creative
3. Allow tools to be combined in any order

On this page

Tool Definitions

1. consult_demogorgon(complaint: str) -> str

- Accepts a complaint string about the Upside Down
- Holds a hardcoded list of 3 possible chaotic/humorous responses
- Randomly selects and returns one response using random.choice()

2. check_hawkins_records(query: str) -> str

- Accepts a search query string
- Holds a hardcoded dictionary of 4 topic-keyed records (portal, monsters, psychics, electricity)
- Iterates through dictionary keys and checks if any key appears in the query (case-insensitive)
- Returns the matching record value, or a default "not found" message if no key matches

3. cast_interdimensional_spell(problem: str, creativity_level: str = "medium") -> str

- Accepts a problem string and an optional creativity_level ("low", "medium", or
- Uses random.sample() to pick N spells (where N = multiplier, capped at list length)
- Returns the selected spells joined by newlines

4. gather_party_wisdom(question: str) -> str

- Accepts a question string
- Holds a hardcoded dictionary of 4 topic-keyed responses attributed to party members (Mike, Dustin, Lucas, Will)
- Iterates through keys to find a match in the question (case-insensitive)
- Returns the matching party response, or a generic "huddle" fallback if no key matches

On this page

Step 3: Create Agent with Tools

Do: Build a LangChain agent that can use these tools flexibly.

Do:

1. Create a prompt that encourages creative problem-solving
2. Set up the agent with tools
3. Create an agent executor

Step 4: Test with Sample Complaints

Do: Test the agent with various complaints.

Step 5: Analyze Agent Behavior

Do: Understand how the agent uses tools creatively.

Do:

1. Track which tools are used
2. Analyze tool chaining patterns
3. Compare with structured approaches

Submission Guidelines

Extension Activities (Optional)

1. **Add more tools:** Create additional themed tools (consult Eleven, check government files, etc.)
2. **Improve tool responses:** Use LLM calls within tools for more dynamic responses
3. **Add memory:** Give the agent conversation memory to reference previous complaints
4. **Create web interface:** Build a simple Streamlit or Gradio interface
5. **Compare performance:** Measure response time and tool usage efficiency

Resources

- LangChain Agents Documentation
- Creating Custom Tools
- Agent Executor Guide

Troubleshooting

Issue: Agent doesn't use tools

- **Solution:** Check tool descriptions are clear, verify prompt encourages tool use, ensure tools are properly registered

Issue: Agent uses tools in wrong order

- **Solution:** This is expected with creative agents! If you need specific order, use LangGraph instead.

Issue: Agent goes into loops

- **Solution:** Set `max_iterations` limit, improve tool descriptions, or add stopping conditions

Issue: Responses are too generic

- **Solution:** Improve tool implementations to return more specific information, adjust temperature

Verify

The reviewer can trace one answer back to retrieved content, tool output, or guardrail evidence.

Explain

Where could the system produce a plausible but unsupported answer?

Submission Sanity Check

Before you submit:

- Keep only files for this lab.
- Name files clearly.
- Remove secrets, API keys, tokens, and unrelated data.
- Make `lab_proof.md` the entry point for the reviewer.

Optional Stretch

On this page

- Add one targeted improvement: better chunking, reranking, a guardrail, a trace view, or a failure-case test.
- Create a small demo UI with vibecoding only if it helps inspect sources and answers. Deploy only if it can run safely without exposed secrets.

LAB | Chaos agent: tools, lore, questionable judgment ○ Not started

Ready to start this lab?

Fork the lab repository, complete the requirements and submit a PR.

Access Github Lab

Have you completed the lab?

Sync lab progress

On this page

Week 4: Day 2